

Boundry: Model-Free Admission and Independently Verifiable Records for AI-Proposed Actions

Ben Mueller

Boundry, Australia

ORCID 0009-0004-0550-1485 verify@boundry.tech

Preprint, version 1.0.1 October 2026

Abstract

As AI systems move from producing text to taking actions, the questions that matter change: what exactly was done, under which rule, what was refused and why, and whether someone independent can check the answers without trusting the operator. In the common *prompt* $\rightarrow$ *model* $\rightarrow$ *action* arrangement, the model’s output is the decision and the history is reconstructed afterwards from logs the operator controls. We describe Boundry, a system in which a model, a person or a program may only *propose* an action in a small declared language. A deterministic kernel that contains no model admits or refuses each proposal against declared rules, executes an admitted proposal from the same sealed bytes that were checked, records refusals as first-class signed outcomes, and appends every step to a signed, hash-chained record. That record can be checked offline by an independent verifier that shares no code with the kernel. We present the design, a threat model, a worked example that readers can reproduce with a public verifier, and an evaluation on synthetic workloads: rebuild of state from the record, detection of single-byte alteration, agreement between independent verifier implementations, and byte-for-byte reproducibility across machines and platforms. The record layer is in live use in two financial applications, the full governing core is demonstrated end to end on a test bench, independent time-stamping is built, and authority-based gating is designed. The contribution is not a new cryptographic primitive but a composition of established ones into a single model-free, fail-closed and independently checkable record of what an AI system was permitted to do and did.

1 Introduction

Much work on AI safety asks how to make a model behave: training, prompting, filtering and monitoring. These approaches matter, but they share a feature: the model’s output remains, directly or through a thin tool layer, the decision to act. When a model only answers questions, a mistake is a wrong answer that can be ignored. When a model acts, by changing a record, moving money or operating a system, a mistake is an effect that has already happened.

This paper takes a complementary position. The harms most often feared from AI agents are harms of *action*, and the safety of an action need not live inside the model. It can live in a small, deterministic component through which every action must pass, that the model cannot reach or persuade, and whose every decision is recorded in a form a stranger can check.

The pattern is old. Proof assistants in the LCF tradition [9], and modern systems such as Lean [6], let anyone propose a proof, a person, a search procedure or an AI model, and trust only a small kernel that checks each step against fixed rules. Security engineering has the related ideas of a reference monitor that mediates every access [2] and of fail-safe defaults [20]. Boundry applies this pattern to *actions* checked against *rules declared by people*, and adds two properties that proofs do not need: the action is executed from the bytes that were checked, and every decision, including every refusal, is recorded in a tamper-evident chain that an independent implementation can verify offline.

Contributions.

1. An architecture in which AI-proposed actions pass through a model-free, deterministic, fail-closed admission step, and are executed from the sealed bytes that were admitted (Section 3).
2. Refusal treated as a first-class, signed, recorded outcome, and a closed verdict vocabulary for verification that separates “could not be checked” and “the checking instrument failed” from “the check failed” (Section 3.4).
3. An explicit threat model stating what is trusted and what is not (Section 2).
4. A public, offline verifier and evaluation kit, so that several central claims can be tested by readers rather than taken on trust, together with an evaluation of rebuild, tamper detection, verifier independence and reproducibility (Sections 4 and 5).

Status. Boundry is being rolled out in stages, and we label each claim with its stage: *in production* (running on real records), *demonstrated* (end to end on a test bench with synthetic data), *built* (implemented, not yet enabled), or *specified* (designed and covered by a patent filing, not yet built). The signed, hash-chained record layer is in live use in a bookkeeping application and, since October 2026, in a financial reconciliation application; each deployment signs with its own key, and independent time-stamping and a rebuild-from-empty on live data are not yet in place. The full governing core (admission, refusal and execution from sealed bytes) is demonstrated; its production deployment is a later stage. Admission by declared operations is demonstrated; gating by recorded authority is specified. Independent time-stamping of checkpoints is built and not yet enabled. Section 8 sets out the stages and the next steps, including independent evaluation.

2 Threat model and trust assumptions

What Boundry is designed to resist.

- **An untrusted author.** The model (or person, or program) that writes a proposal may be mistaken, manipulated by injected content, or adversarial. Boundry assumes nothing about the author’s intent.
- **Divergence between what was approved and what was done.** An action must not be executed from anything other than the bytes that were admitted.
- **Silent after-the-fact alteration.** A change to any recorded entry must be detectable by a party who did not produce it.
- **A verifier that inherits the producer’s defects.** Verification should not depend on the producer’s own code.

What is trusted.

- The kernel’s code and the integrity of the host on which it runs at the time of execution.
- Custody of the signing keys, and the distribution of the corresponding verifying keys to readers. In the public kit the verifying key ships with the kit itself, so a reader must trust that the key is the operator’s.
- The people who declare the rules. Boundry applies the rules it is given; it does not judge whether they are good.
- The edge connectors that carry out an admitted effect (writing a record, sending a message), to the extent that they faithfully apply the sealed bytes they are given.

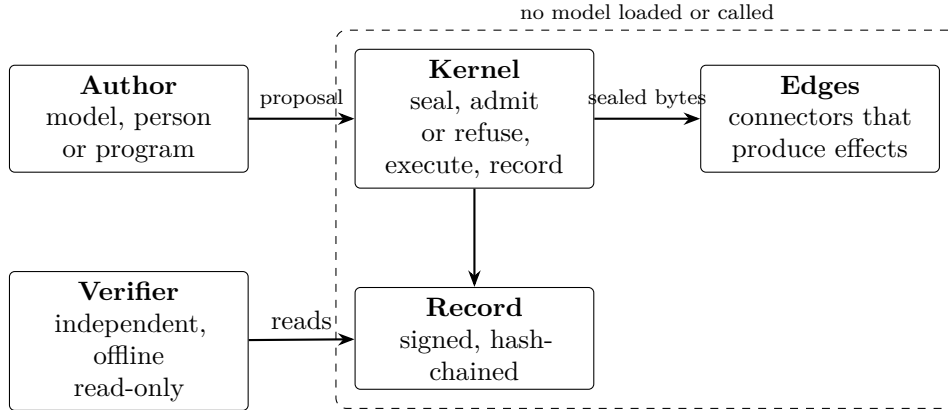


Figure 1: The author may only propose. The kernel is the only component that can authorise an effect; edges act only on admitted, sealed bytes. The verifier shares no code with the kernel and needs only the record and a verifying key.

Out of scope. Actions that reach the world by a route that does not pass through the kernel; compromise of the host or keys; the truth of inputs (Boundry checks that a declared process was followed, not that its inputs were correct); availability and denial of service; and side channels.

External anchoring. The record is *tamper-evident*: an edit is not prevented, but it cannot pass unnoticed. Because a single operator holds the signing key in current deployments, an external witness is what rules out an operator rewriting and re-signing a whole chain. Boundry is designed to provide this by anchoring signed checkpoints with an RFC 3161 time-stamping authority [1], with a transparency log [13] as an alternative. The mechanism is built; once it is enabled on production checkpoints, the absence of rewriting becomes independently checkable rather than resting on the operator.

3 Design

3.1 Three parts and one boundary

Boundry separates three roles (Figure 1). The *author*, typically a model, reads a request and writes a proposal. The *kernel* decides whether the proposal may run, runs it, and records both. The *edges* are ordinary connector code that produce real-world effects, driven only by admitted, sealed proposals. A fourth component, the *verifier*, sits outside the system entirely and reads the record.

The kernel is ordinary, deterministic software. It contains no model and cannot reach one: the governed code path has no route by which a model can be loaded or called. “No model in execution” does not mean “no code in execution”; it means the code that decides and records contains no model.

3.2 The life of a request

Proposal. The author expresses an intended action in a small declared language: which operation, on what, on whose behalf. Requests in this language are finite, bounded, loop-free plans composed from a registered set of operations (*primitives*). The language is deliberately not general-purpose. A proposal is only a suggestion; nothing has happened.

Seal. The proposal is reduced to a canonical byte form, fingerprinted, signed, and bound to the requester and time. From this point it cannot be changed without detection.

Admit or refuse. The kernel checks the sealed proposal against fixed rules: that it is a known, well-formed kind and that only permitted primitives appear. Anything not declared is refused. Reads and writes are scoped to a declared tenant. Admission extends to gating actions by recorded authority, with a verdict recorded before the effect; this stage is specified (designed, not yet built), and we make no evaluation claim for it. Admission does not consult the author’s reasoning, so injected text can change what is *proposed* but cannot make a proposal pass that the declared rules do not permit. This follows from the architecture; testing it against an adversarial prompt-injection benchmark is a planned next step.

Execute from the sealed bytes. An admitted proposal runs inside declared time, memory and work limits. Where an effect is produced at an edge, the edge is driven from the sealed proposal, never from the author’s live output, so what was checked and what was done are the same bytes. On the bench, execution reads the sealed proposal and the record binds proposal, decision and result; edge connectors for external effects belong to the production deployment of the core, a later stage.

Record. Every stage is sealed and appended to a chain in which each entry commits to the digest of its predecessor [5, 21]. Periodically the kernel signs a checkpoint over the record set, committing to its size and a Merkle-style root [15]. Checkpoint production is signalled from the append path at constant cost; deriving, signing and any external anchoring sit off that path, so an external party’s outage cannot stall a seal.

3.3 Properties of the construction

- **A canonical form that refuses ambiguity.** Two conforming implementations produce identical bytes for the same value. A value with two possible renderings is refused and named rather than approximated: floating-point numbers, keys that collide after Unicode normalisation, an absent field conflated with a null one. Numeric determinism comes from integers and fixed-point decimals. (Compare JCS [19], which also rejects some values, such as non-finite numbers, but otherwise serialises numbers in a defined floating-point form.)
- **Identity independent of the signer.** What a record *is* and who *vouched for it* are separate. Identical work under different keys has one identity, and key rotation does not rewrite the past. Keys rotate by epoch; retiring a key is distinguished from revoking it.
- **Absence is explicit.** “No predecessor” is expressed by absence, not by a sentinel value that a reader might mistake for a real value or for failure. More generally, the design refuses and names an absent precondition rather than defaulting through it.
- **Content-addressed storage.** Record bodies are addressed by their hash, and an identifier cannot be rebound to different bytes.
- **Determinism as a measured property.** The runtime is declared, and undeclared non-deterministic dependencies are sought by perturbation rather than assumed absent. The cross-platform and clean-room results in Section 5 are the measurements we report.

3.4 Refusal and the verdict vocabulary

A refusal is not an error, a retry or a silent default. It is a signed record with a stated reason, kept beside admissions. In the public evaluation kit, for example, a proposal that names a primitive outside the permitted set is refused at the resolution stage with the reason `primitive_not_in_allowlist`, and that refusal is itself a verifiable record.

Verification uses a closed vocabulary (Table 1). The INDETERMINATE verdict is marked draft in the released verifier. Comparison of two runs is reported separately, as EQUIVALENT,

Table 1: Verification verdicts, in order of precedence (highest first).

Verdict	Means	Does not mean
INDETERMINATE	The verifier failed to complete.	Anything about the record.
ALTERED	A valid attestation exists and the content no longer matches it.	Who altered it, or when.
REFUTED	Evidence was supplied and failed to verify.	Forgery or bad faith; corruption, a wrong key or truncation produce the same verdict.
UNATTESTED	Nothing attests to these bytes.	That the record is false.
ATTESTED	The named check ran and passed.	Anything beyond the stated scope.

DIVERGENT or UNREACHABLE. Each verdict states both what it means and what it does not mean. Two distinctions matter most. First, *absence of attestation is not falsity*: a record with nothing attesting to it is UNATTESTED, never “false”. Second, *failure of the instrument outranks any finding about the record*: if the verifier itself cannot complete, the result is INDETERMINATE and says nothing about the record either way.

3.5 Independent verification

The verifier, Boundry Verify [17], is a separate, read-only package that runs on the reader’s machine using only the Python standard library. It opens no socket, cannot sign, writes nothing, and attaches to no database; each of these properties is held by a named self-check with a planted control that the check must reject. It re-derives fingerprints, recomputes plan identities, walks each link from submitted intent to output, and compares runs by hash rather than by re-execution. Because it shares no code with the kernel, a defect in one is unlikely to be masked by the same defect in the other.

4 A worked example readers can reproduce

The public evaluation kit [17] contains real governed runs of the kernel on synthetic inputs: a calculation, a small transformation chain, and one refused request, each executed in fresh processes on macOS arm64 and on Linux aarch64. It also contains three deliberately tampered copies of one run (a forged hash, a changed output byte, a lost output) and one verifying key. Each run ships with its submitted intent beside it, because the record carries the intent’s hash rather than the intent.

A reader with Python 3.12 or later can install the verifier and ask three questions:

1. *Compare the calculation run on macOS with the same run on Linux.* The result is EQUIVALENT. Outputs, plan body, result and payload signatures are byte-identical. The fields that differ are wall-clock stamps and hashes taken over them, together with descriptive metadata (the recorded environment and platform, and the run label); each is named and marked as expected.
2. *Compare the macOS run with its forged-hash copy.* The result is DIVERGENT: the forgery is caught by hash comparison, and the failing envelope is named.
3. *Explain the transformation run.* The verifier walks the lineage from submitted intent to output, recomputing each link offline.

The refused run can be inspected in the same way: it is a signed record whose reason names the primitive that was not permitted. None of these steps requires access to the kernel or a network connection. They rely on the verifying key shipped with the kit. The kit demonstrates the refusal side of admission; a kit showing authority-based gating will follow that stage.

Table 2: Evaluation summary. “Bench” means synthetic data on the developer’s machines; “kit” means the public evaluation kit; “production” means live use on real records.

Property	Method	Result
Test suites (bench)	Kernel suite; corpus gate; full programme test set on the primary machine	2,199 passed, 0 failed; 283 passed; 3,603 passed, 6 skipped
Rebuild from record (bench)	12 requests producing 144 sealed envelopes; state rebuilt from the record alone and compared	Identical; 144 form hashes re-derived; source unchanged
Tamper detection (bench)	One byte altered in a copy of a sealed record	Refused; altered body and both hashes named
Checkpoints (bench)	All checkpoints verified; heads recomputed from leaves; consecutive pairs checked for consistency; equivocation check	144 of 144 verified and recomputed; of 143 consecutive pairs, 99 advanced and 44 repeated an unchanged checkpoint; none went backwards; no equivocation
Verifier independence (bench)	Kernel’s own verifier vs. offline verifier on the same 144 bundles	Agree on 144 of 144; witness receipt UNATTESTED on all, as no witness exists
Second implementation (bench)	A separate Go verifier written from the specification, not ported, vs. the Python verifier	Agree on 148 of 148 vectors (internal; not published or third-party)
Clean-room rebuild (bench)	Second machine; only two sealed source bundles and an offline cache of 57 packages; network blocked	Source trees, commit identities and an engine digest over 175 files equal to the primary machine’s published figures
Cross-platform determinism (kit)	Same governed requests on macOS arm64 and Linux aarch64, by different people	Byte-identical except wall-clock stamps, hashes over them, and descriptive run metadata
Rebuild throughput (bench)	Rebuild from empty, one machine	100,000 records in 9.27 s
Verifier self-checks (kit)	Package’s own checks, standard library only	78 run, 0 failures, 0 skipped (Python 3.12, 3.13)
Record layer (live use)	Every step sealed and recorded; in the reconciliation application, the application checks daily that its records match the chain and holds approvals on any mismatch	In live use in a bookkeeping application and, since October 2026, in a financial reconciliation application; each signs with its own key

5 Evaluation

Table 2 summarises what has been measured. Unless stated otherwise, results come from a test bench with synthetic data and freshly generated keys.

On the checkpoint row: in 44 of the 143 consecutive pairs, the later checkpoint repeated the earlier one at the same tree size, so there was no extension for a consistency proof to cover; in the other 99 the tree advanced and the consistency proof held, and no pair went backwards. The run’s separate equivocation check reported none, every checkpoint head was recomputed from the leaves (144 of 144), and both planted consistency controls fired as expected.

Four qualifications apply. First, on the clean-room machine the full programme test set reported 11 failures (3,594 passed, 4 skipped); each was traced to an environmental difference (interpreter path, files outside the repositories, file timestamps, test harness) rather than to the software, and we report them rather than omit them. Second, the two reproducibility results are distinct: the clean-room rebuild shows that the *software* rebuilds identically on another machine, while the kit shows that the same *governed requests* execute identically on two platforms. Third, the rebuild figure is a throughput measurement on one machine, not a per-request latency; per-request latency will be measured in realistic agent workloads, where we expect it to be small

relative to model inference time. Fourth, the kit results can be reproduced by readers today; the bench results, including the second verifier implementation, are reported from internal runs, and we would welcome independent replication.

Development method. The system was built with AI-assisted engineering tools, organised into roles and constrained by the substrate’s own checks: canonical forms, invariants, reproducibility gates and the test suites above. Proposed changes that failed these checks were refused and recorded, and many of the system’s primitives were introduced because an earlier proposal was refused for lacking them. We regard this as an experience report, not evidence that the method is better than conventional engineering; it is described further in [16].

6 Related work

Runtime guardrails and enforcement outside the model. Shamsujjoha et al. [22] give a taxonomy and reference architecture for layered guardrails; in its terms, Boundry is a rule-based, fail-closed guardrail at the action stage that also produces a verifiable record of its own decisions. CaMeL [7] enforces data-flow policies on a model-written program through a custom interpreter and tracks the provenance of values, which Boundry does not. Progent [23] applies deterministic privilege policies to tool calls and is among the closest designs; Boundry differs in executing from the checked bytes and in producing an independently verifiable record of each admission and refusal. AgentSpec [25] enforces trigger-check-enforce rules at low overhead; its rules fire on matching triggers, whereas Boundry refuses anything not declared. FIDES [4] applies information-flow control to agents, and Beurer-Kellner et al. [3] catalogue design patterns against prompt injection. Boundry is complementary to these: it is a candidate for the enforcement-and-evidence layer beneath them rather than a replacement.

AI control. Greenblatt et al. [10] study protocols that remain safe against an intentionally subversive model. Boundry is a trusted, model-free permission and record layer of the kind such protocols can sit on; it is not itself a control protocol, and evaluation against an attack policy is a natural next step.

Auditing agent actions and multi-agent risk. Tracekit [8] gates a coding agent’s tool calls with a pre-execution policy and records intent, reasoning and action in a tamper-evident, externally anchorable ledger. It is the closest work on the evidence side. Boundry differs in making admission model-free and limited to declared operations, in executing from the sealed bytes that were admitted, and in verification by an implementation independent of the producer. Reid et al. [18] analyse risks and controls for agents deployed across organisational boundaries; Boundry’s current deployments operate within one organisation, and cross-organisational operation is a later stage.

Tamper-evident logs, transparency and supply chains. Hash-chained audit logs [21], efficient tamper-evident log structures [5], Merkle commitments [15], certificate transparency [12, 13], trusted time-stamping [1], reproducible builds [11] and supply-chain attestation [24] are established. None is claimed here as new.

Rules as code. Catala [14] and LLM-assisted formalisation of tax law [26] treat human-declared legal rules as a deterministic authority. Catala computes *from* the rules; Boundry governs who may propose an action *under* them and records each decision verifiably. The two could be combined.

Kernels and reference monitors. The small trusted checker follows the LCF tradition [6, 9]; the requirement that every access be mediated follows the reference-monitor concept [2] and the principle of fail-safe defaults [20].

What is new. The contribution is the composition: an author that may only propose and a model-free deterministic kernel that disposes; execution from the admitted bytes; identity independent of the signing key; refusal as a named, signed, first-class outcome; verification by an independent implementation, offline, with a verdict vocabulary that distinguishes instrument failure and absence of attestation from refutation; and a design organised around refusing and naming an absent precondition rather than defaulting through it.

7 Design boundaries

These are properties of the design itself, as distinct from deployment stage. They state what a kernel of this kind is for, and what it deliberately leaves to other layers.

- **Coverage follows mediation.** Boundry protects the actions that pass through it. Deployments are designed so that the kernel is the sole path to consequential actions.
- **Permitted is not the same as intended.** If the author misunderstands a request and proposes something the rules allow, it is admitted. Consequential actions should therefore include a human confirmation step.
- **The rules carry the judgement.** The kernel moves the trust problem from the model to the rules, where it is explicit, inspectable and attributable to the people who wrote them.
- **Process, not truth.** The record shows that a declared process was followed and by whom, not that its inputs were true. A signature establishes origin and integrity.
- **Complementary to information-flow and trajectory analysis.** Each proposal is judged against declared rules; value-level provenance tracking and multi-step trajectory analysis, as in some of the systems in Section 6, can sit alongside it.
- **Bounded by design.** Requests are finite, loop-free plans over registered primitives. This keeps every request checkable before it runs; new capability is added by registering primitives.

8 Deployment stages and next steps

We particularly welcome independent evaluation: the public verifier and kit are intended to make that straightforward.

9 Beyond AI

The underlying problem is older than AI: an untrusted author whose output becomes action, whether a script, an integration or a junior employee. Nothing in the kernel requires the author to be a model. A system built this way could move the decision logic of a conventional backend (request validation, authorisation and tenant separation, business rules and the audit trail) into a small declared set that the kernel checks and records, while interfaces and integrations remain ordinary code. Measuring how far this reduces the size and opacity of real codebases is an open and, we think, interesting question.

Capability	Stage	Next step
Signed, hash-chained record layer	Live use	Independent time-stamping; rebuild-from-empty on live data
Admission, refusal, execution from sealed bytes	Demonstrated	Production deployment (later stage)
Independent time-stamping of checkpoints	Built, not enabled	Enable on production checkpoints when decided
Gating by recorded authority	Specified	Implementation (not yet scheduled)
INDETERMINATE verdict	Built (draft)	Finalise in a verifier release
Per-request overhead	Planned	Measure in realistic agent workloads
Adversarial and independent evaluation	Planned	Prompt-injection benchmark; invite external red-teaming
Cross-organisation operation	Planned	After single-organisation deployments mature
Formal proofs of core properties	Planned	Mechanise key invariants

10 Conclusion

Boundry relocates trust for AI-driven actions from the model to a small, deterministic, model-free kernel, and makes the kernel’s decisions checkable by anyone with an offline verifier. The construction is assembled from established parts. What we offer is a working composition, an explicit statement of its trust assumptions and deployment stages, and public artefacts that allow readers to test its central claims for themselves. The most useful next steps are independent evaluation, adversarial testing, external anchoring of checkpoints, and measurement of overhead in realistic agent workloads. We welcome researchers who wish to examine, test or challenge the claims made here.

Availability

A read-only offline verifier, its source and the evaluation kit are available at <https://github.com/Boundryos/boundary-verify> and <https://verify.boundary.tech>. The verifier is free to use and is not open source. Research notes, a claims-and-evidence table and essays are at <https://github.com/Boundryos/research> (DOI 10.5281/zenodo.23129907). The kernel is proprietary. The methods described are the subject of Australian provisional patent applications; no patent licence is granted by this paper.

Use of AI tools

AI-assisted tools were used in developing the software described and in drafting this paper. The author reviewed all content and is responsible for it.

References

- [1] Carlisle Adams, Pat Cain, Denis Pinkas, and Robert Zuccherato. Internet X.509 public key infrastructure time-stamp protocol (TSP). RFC 3161, 2001.
- [2] James P. Anderson. Computer security technology planning study, volume II. Technical Report ESD-TR-73-51, Electronic Systems Division, US Air Force, 1972.
- [3] Luca Beurer-Kellner, Beat Buesser, Ana-Maria Crețu, et al. Design patterns for securing LLM agents against prompt injections, 2025. arXiv:2506.08837.
- [4] Manuel Costa, Boris Köpf, Aashish Kolluri, et al. Securing AI agents with information-flow control, 2025. arXiv:2505.23643.

- [5] Scott A. Crosby and Dan S. Wallach. Efficient data structures for tamper-evident logging. In *18th USENIX Security Symposium*, 2009.
- [6] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.
- [7] E. Debenedetti, I. Shumailov, T. Fan, et al. Defeating prompt injections by design, 2025. arXiv:2503.18813.
- [8] Bravish Ghosh. Tracekit: Tamper-evident intent-reasoning-action auditing for autonomous coding agents, 2026. arXiv:2609.35659.
- [9] Michael J. Gordon, Arthur J. Milner, and Christopher P. Wadsworth. *Edinburgh LCF*, volume 78 of *Lecture Notes in Computer Science*. Springer, 1979. doi: 10.1007/3-540-09724-4.
- [10] Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. AI control: Improving safety despite intentional subversion. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2312.06942.
- [11] Chris Lamb and Stefano Zacchiroli. Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software*, 39(2):62–70, 2022. doi: 10.1109/MS.2021.3073045.
- [12] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. RFC 6962, 2013.
- [13] Ben Laurie, Eran Messeri, and Rob Stradling. Certificate transparency version 2.0. RFC 9162, 2021.
- [14] Denis Merigoux, Nicolas Chataing, and Jonathan Protzenko. Catala: A programming language for the law. *Proceedings of the ACM on Programming Languages*, 5(ICFP):1–29, 2021. doi: 10.1145/3473582.
- [15] Ralph C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology – CRYPTO ’87*, volume 293 of *Lecture Notes in Computer Science*, pages 369–378. Springer, 1988. doi: 10.1007/3-540-48184-2_32.
- [16] Ben Mueller. Boundry: research notes, claims and evidence, and development history. Zenodo, <https://github.com/Boundryos/research>, 2026. DOI 10.5281/zenodo.23129907 (all versions).
- [17] Ben Mueller. Boundry Verify: a read-only, offline verifier and evaluation kit. <https://github.com/Boundryos/boundary-verify> and <https://verify.boundary.tech>, 2026. Version 0.3.1.
- [18] A. Reid, S. O’Callaghan, D. Venini, L. Carroll, and T. Caetano. Risks and controls for multi-agent systems: An analytical framework for deployment of AI agents across organisational boundaries, 2026. Published by the Australian AI Safety Institute. arXiv:2608.26626.
- [19] Anders Rundgren, Bret Jordan, and Samuel Erdtman. JSON canonicalization scheme (JCS). RFC 8785, 2020.
- [20] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi: 10.1109/PROC.1975.9939.
- [21] Bruce Schneier and John Kelsey. Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security*, 2(2):159–176, 1999. doi: 10.1145/317087.317089.
- [22] Md Shamsujjoha, Qinghua Lu, Dehai Zhao, and Liming Zhu. Swiss cheese model for AI safety: A taxonomy and reference architecture for multi-layered guardrails of foundation model based agents. In *IEEE 22nd International Conference on Software Architecture (ICSA)*, pages 37–48, 2025. doi: 10.1109/ICSA65012.2025.00014. arXiv:2408.02205.
- [23] T. Shi, J. He, Z. Wang, et al. Progent: Securing AI agents with privilege control, 2025. arXiv:2504.11703.
- [24] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium*, 2019.

- [25] H. Wang, Christopher M. Poskitt, and Jun Sun. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. In *Proceedings of the 48th International Conference on Software Engineering (ICSE)*, pages 2938–2950, 2026. doi: 10.1145/3744916.3764546. arXiv:2503.18666.
- [26] B. Yadamsuren, S. K. Platt, and M. Diaz. LLM-assisted formalization enables deterministic detection of statutory inconsistency in the Internal Revenue Code, 2025. arXiv:2511.11954.